

Lessons Learned – Vikas Neriyanuru

CS6650 Final Project

April 2026

What I worked on

My primary areas were the `flash-sale-api` service (atomic DECR design, quantity support, and the Experiment 2 strategy selector), Experiment 1 (queue fairness), Experiment 2 (inventory correctness), and the local parity stack that let us run experiments without burning ECS time.

What went well

The local-dev harness paid for itself. Spinning up Redis + LocalStack via docker-compose with a one-command bootstrap of SQS and DynamoDB tables turned “days of ECS iteration” into “minutes of local runs.” The AWS SDK v2’s `AWS_ENDPOINT_URL` support means no code changes were needed – the same binary runs against LocalStack in dev and real AWS in prod.

The atomic DECR design held up. Experiment 2’s 200-concurrent-buyer run returned exactly 100 201 Created + 100 409 Conflict, zero oversell, across three buyer scales and three strategies. That is the distributed-systems concept we covered early in the course – a single atomic operation beats read-check-write every time – and it was validated with real data, not just in theory.

What went wrong

I merged code that shipped a silent bug. The Experiment 2 quantity-support PR added `DecrInventoryBy` + `rollback-IncrBy`. The design was correct and the tests passed, but the Experiment 1 sweep two weeks later revealed that the waiting-room’s `timestamp_incr` tiebreaker was a no-op due to float64 precision loss. That bug was in the service I didn’t own – but my experiment was the one that found it. A contract test or an early micro-benchmark would have caught it day one.

I never stress-tested beyond the proposal’s 1k-user ceiling. On submission day I bumped each experiment to 10,000 concurrent users as a gut-check. Correctness held – zero oversell, zero position collisions, SQS drained – but the waiting-room started returning `redis: connection pool timeout` on its admission-ticker’s background `INCR`. The go-redis default `PoolSize` of `10 * NumCPU` (about 80 on my laptop) was starved by the user-facing burst at ~1,700 rps, and the metric ticker lost connection-wait races. Two-line fix (`PoolSize: 2000`, `PoolTimeout: 10s`). The lesson: our unit of concurrency thinking was wrong. We designed for “many users” but measured at a load where library defaults still sufficed. Real flash sales are a 100x burst against steady-state, and the defaults of every tier – HTTP client, Redis client, connection pool, DNS cache – have to be re-examined against that shape of load, not the average.

Locust vs. Go harness. The proposal specified Locust for the experiments. On the day of the experiments Locust was not installed, and I pivoted to a Go-native harness. That worked – all three experiments ran and produced clean CSVs – but the pivot cost an hour I didn’t budget for.

How the course concepts applied

- Atomic operations vs. distributed locks. Experiment 2’s optimistic-locking row (sells 17/100 units under contention) directly demonstrates the failure mode we discussed around WATCH/MULTI/EXEC: correctness is preserved but availability collapses. Atomic DECRBY, by contrast, uses Redis’s single-threaded serialization as the coordination mechanism – cheaper *and* more available.
- Compensating transactions. The DECR-then-IncrBy-on-SQS-fail pattern in flash-sale-api is a classic compensating transaction: when the downstream publish fails we restore the inventory to keep the system consistent. The alternative – a distributed 2PC across Redis and SQS – would be far more expensive for the same invariant.
- Message-driven decoupling. Shifting persistence off the critical request path into SQS + order-worker is what let Experiment 3’s admission-rate sweep find peak SQS depth = 0 across all rates tested: the API can accept bursts that the database layer could never handle synchronously.

What I would do differently

1. Run each experiment end-to-end on day one, even with 10 users. Five-minute smoke runs would have surfaced both the float64 bug and the missing admission-token gate two weeks earlier.
2. Add service-to-service contract tests. The waiting-room and flash-sale-api each had unit tests that were correct in isolation. The missing integration test is what let the admission-token gap ship.
3. Pin the Go version. The `tests/flash-sale-api/go.mod` quietly drifted to go 1.25.5; CI would have caught it, local dev did not.